

AOS v2 — Buildable Master Specification

Self-hosted, FOSS-only autonomous engineering system.
Revision incorporating pre-build architecture audit (memory/hallucination layer, model-tier reality, network security).

Prepared 10 September 2026 · Supersedes AOS Quality-First Rewrite §2–16 where noted

1. Purpose & Non-Negotiables

AOS is a controlled, self-hosted autonomous software-engineering machine. Given an objective, it researches, plans, implements, tests, debugs, reviews, and hardens code in a durable loop until verified evidence supports completion — or a human decision is required. It is not a chatbot, not an editor plugin, and it does not claim completion without evidence.

Guarantee	Meaning
No false completion	PASS is derived from required checks against the exact commit, never from an agent's self-report.
Bounded autonomy	The system acts without asking, but only inside a policy engine it cannot override.
Recoverability	Worker, provider, or network failure preserves state; work resumes, it does not restart from zero.
Replaceability	Frontend, model provider, and compute worker can each be swapped without touching core contracts.
Honest uncertainty	"Guaranteed bug-free software" is not promised. Residual uncertainty is reported, not hidden.

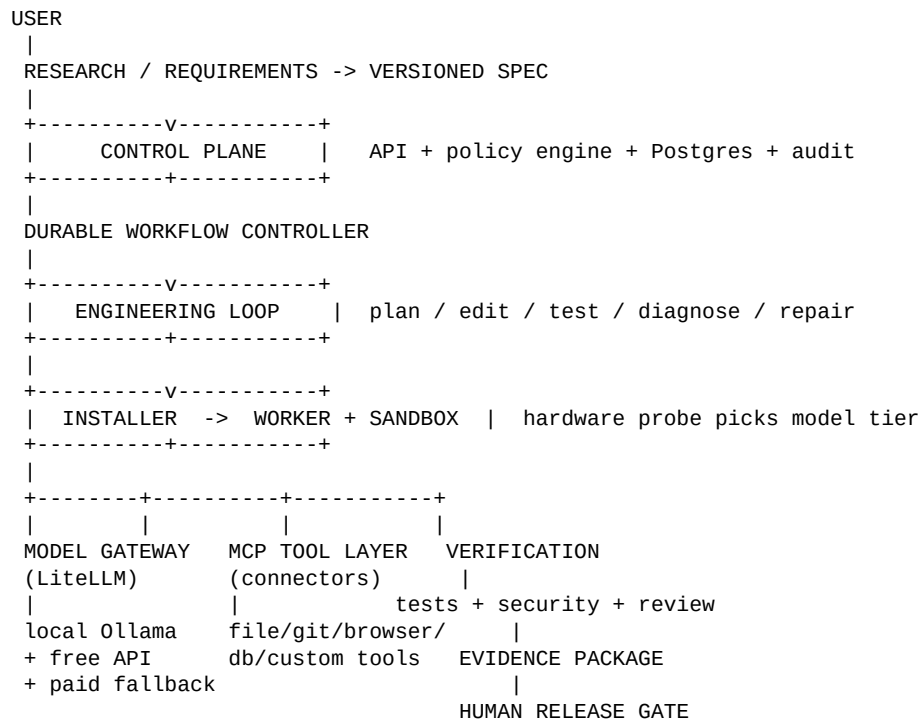
Ownership boundary: one human operator holds release authority. The system exercises maximum autonomy on routine engineering work; it holds zero authority over production, spend, secrets, or external systems outside an explicit policy grant.

2. Scope Lock — v1

Cutoff rule: a component ships in v1 only if it directly improves correctness, repair, security, durability, observability, or human control. Everything else is deferred, not abandoned.

KEEP — v1	DEFER — later phase
Sandboxed execution (Docker, ephemeral)	Kubernetes
Git + task branches	Terraform
One durable workflow controller	Multiple orchestration frameworks
One strong coding agent, tier-routed	Large multi-agent swarm
Model/provider gateway (LiteLLM)	Dozens of providers
Deterministic validation	“Quality score” heuristics
Self-repair loop + repair-event log	Infinite/unbounded retry
Independent review pass	Many specialist review agents
PostgreSQL	Redis (unless real load proves need)
Typed event stream	Elaborate UI polish
Human approval gates	Full multi-tenancy
Basic structured memory (Postgres-first)	Custom vector database
Benchmark / eval suite	Broad ECC integration at the start
Hardware-detect installer (Phase 2, small)	Automatic cloud fleet management
MCP as tool-interface standard (Phase 3)	Bespoke plugin marketplace
Headscale/WireGuard mesh for remote workers	Any inbound open port — rejected outright, not deferred

3. Reference Architecture



REMOTE WORKER (optional, Phase 11):
CONTROL PLANE <==Headscale/WireGuard mesh==> REMOTE GPU WORKER
(no inbound open port, ever – mesh membership only)

FRONTEND = replaceable client of backend, never source of truth.

4. Pinned Component Decisions

Layer	Choice	Why
Frontend	Next.js + TypeScript, @xyflow/react	Replaceable client; graph is a projection of backend events.
Backend	FastAPI + Pydantic v2	Typed contracts, async-native.
Database	PostgreSQL	Single source of truth; vector only if proven necessary.
Workflow controller	OPEN — Temporal (durable, heavier ops) vs. custom asyncio+Postgres controller (lighter, less battle-tested). Decide in Phase 1 against your actual solo-ops tolerance.	Both satisfy “recoverability”; the trade is operational complexity vs. proven durability.
Agent loop	One coding-agent loop, model-tier routed	No swarm until one task completes reliably end to end.
Model gateway	LiteLLM proxy (OSS)	One interface over local Ollama + free API tiers + paid escalation.
Local inference	Ollama	Tag selected by Installer's hardware probe.
Tool interface	MCP servers (official Python SDK)	Open protocol — satisfies “connectors/plugins” without vendor lock-in.
Sandbox	Docker, ephemeral per task, rootless where possible	Isolation without Kubernetes overhead.
Installer	Python probe (“aos-init”)	Detects GPU/VRAM/RAM/CPU, writes config, pulls matching Ollama tag.
Artifacts	MinIO (S3-compatible)	Self-hostable object store.
Secrets	OpenBao (open-source Vault fork)	Just-in-time, narrow-scope secret issuance.
Telemetry	OpenTelemetry, or plain structured logs in v1	Upgrade path without re-instrumenting.
Remote compute network	Headscale + WireGuard mesh	Encrypted mesh membership — zero inbound open ports.
Code truth	Git	Task branches; PR/patch is the unit of work.

5. Data & Event Contracts

Object	Minimum contents
Task	objective, requirement version, acceptance contract, policy, status
Run	task, workflow version, worker, model route, timestamps
Artifact	type, producer, URI, SHA-256, run
Event	ID, run ID, sequence, type, timestamp, actor, payload
Approval	action, evidence, actor, scope, expiry, decision
Policy	version, principal / action / resource rules
Worker	capabilities, health, lease, cost metadata

Canonical events: run.state.changed, agent.started, tool.requested, tool.completed, test.completed, review.finding, approval.requested, approval.decided, run.failed, run.completed, **repair.attempt** (new — failure_class, hypothesis, patch_sha, result; closes the audit gap where a repair loop could mutate output with no traceable record).

6. Autonomous Engineering Loop & Self-Repair

TASK CONTRACT: objective + acceptance criteria + allowed scope
+ repo snapshot + constraints + required checks

|
v
OBSERVE -> PLAN -> EDIT -> FOCUSED TEST -> PASS? --yes--> EXPAND TEST -> INDEPENDENT REVIEW -> EVIDENCE PACKAGE
|no
CLASSIFY FAILURE -> DIAGNOSE/REPAIR (emits repair.attempt) -> back to FOCUSED TEST

Failure class	Response
Code defect	Reproduce → localize → repair → rerun
Test defect	Inspect assumptions → repair test, or escalate if contract changed
Environment defect	Separate environment failure from code failure
Flaky test	Rerun within a flake budget; never silently pass
Tool failure	Retry/fallback → another safe route
Model failure	Reframe task, reduce context, or switch model tier if policy permits
Requirement conflict	Stop affected work, request a decision
Security/policy failure	Never bypass — escalate or use an approved alternative

Hard rule (new): every self-repair attempt **MUST** emit a `repair.attempt` event before the next action executes. A repair cycle with no corresponding event trail is a policy violation, not a bug — it is exactly the “silent second pass” failure mode that makes agent systems unauditably.

7. Verification Is the Quality Gate

Check	Purpose
Requirement coverage	Confirm every acceptance criterion is addressed
Unit/integration tests	Validate local and system behavior
Build/type/lint	Validate structural correctness
Behavioral/E2E	Validate realistic user flows
Security	Secrets, dependencies, authZ, unsafe behavior
Regression	Previously solved behavior remains intact
Independent review	Missing cases, bad assumptions, unnecessary changes
Environment binding	Evidence belongs to the exact commit/environment

PASS rule: the backend transitions a task to READY only when every required check for that task contract has valid evidence bound to the exact code state. “The agent says done” is never sufficient.

8. Security & Authority

Action	Default
Read project repo	Allowed in project scope
Write code	Task branch / workspace only
Terminal	Sandboxed + resource/command policy
Network	Deny by default; allow required domains
Inbound ports	NONE — no exceptions. Remote workers join via Headscale/WireGuard mesh only.
Secrets	Just-in-time, narrow scope, redacted
Protected merge	Human approval
Production deploy	Human approval
Paid compute	Budget + approval
Destructive operation	Human approval

Invariant: an agent may request an action; only the policy engine — never the agent — decides whether it executes.

9. Multi-Session / Multi-Chat-Window Build Protocol

No single chat context can hold this entire build. AOS itself is built the way it will later build software: as a sequence of bounded task contracts handed between independent execution sessions, each starting cold.

Phase Packet — one markdown file per build phase, self-contained: role framing, condensed context recap, objective, tech decisions relevant to that phase only, explicit deliverables, explicit out-of-scope boundary, acceptance/exit criteria, validation commands, and a required Handoff Block template.

STATE.md — one running file the human operator carries between chat windows. Each phase's executing agent appends its filled-in Handoff Block on completion: files created, commands run, exit evidence, decisions made, open questions for the next phase.

Session start protocol — a new chat window is seeded with: (1) this Master Specification or the relevant excerpt, (2) STATE.md so far, (3) the next Phase Packet. The agent must read STATE.md and confirm the prior phase's exit criteria are actually met before doing any work. If they are not met, it stops and says so instead of guessing forward — the same no-false-completion rule AOS itself will enforce on code. File manifest and phase status live in 00_BUILD_INDEX.md, delivered alongside this specification.

10. Final Build Order

#	Phase	Exit condition	Packet file
0	Contracts + threat model	Authority/state rules explicit	PHASE_00 (delivered)
1	FastAPI + PostgreSQL + workflow decision	State survives restart	PHASE_01 (pending)
2	Installer + Worker + sandbox	Code runs safely in isolation on the detected hardware tier	PHASE_02 (pending)
3	Model gateway + MCP tool layer	Two providers execute the same task contract via MCP tools	PHASE_03 (pending)
4	Single coding loop	Toy issue → branch → patch → tests	PHASE_04 (pending)
5	Verification	Broken patch cannot become PASS	PHASE_05 (pending)
6	Self-repair + repair-event log	Seeded failure repaired automatically, fully logged	PHASE_06 (pending)
7	Independent review	Reviewer can reject implementation	PHASE_07 (pending)
8	Event stream + graph UI	UI mirrors backend and reconnects	PHASE_08 (pending)
9	Research + requirements	Evidence → questions → locked spec	PHASE_09 (pending)
10	Memory	Validated knowledge retrieval	PHASE_10 (pending)
11	Remote worker + Headscale/WireGuard mesh	Same worker contract works remotely, zero open ports	PHASE_11 (pending)
12	Release gates	Consequential action cannot bypass approval	PHASE_12 (pending)
13	Hardening	Fault/security/regression suites block bad versions	PHASE_13 (pending)

Phases are strictly sequential for v1. Do not start a phase whose predecessor's exit condition is not evidenced in STATE.md.

11. Certainty Notes

[Certain] Sections 1, 5, 6, 7, 8 are carried over from the source AOS Quality-First Rewrite with no substantive change — that reasoning already held up. [Certain] Sections 2, 3, 4 are revised: installer, MCP, and the mesh-only networking rule are additions; the open-port pattern is rejected outright rather than deferred. [Likely] the Temporal-vs-custom-controller decision in §4 is genuinely open and belongs to Phase 1, not this document — don't let Phase 0 stall on it. [Guessing] exact phase durations — no benchmark data exists yet; Phase 13's own suite is what will produce that data.